Document 11

Delivery and Acceptance Criteria

1. Purpose of This Document

This document defines the required deliverables, review conditions, and acceptance criteria for the Trinity Engine v1 implementation.

Its purpose is to specify:

- what the engineer must deliver
- what constitutes a complete implementation
- what evidence must be provided for review
- how canon compliance is assessed
- how defects, ambiguities, and canon conflicts are handled during implementation
- when the implementation may be accepted
- when the implementation must be rejected or returned for correction

This document turns the canon package into an enforceable implementation handoff.

---

2. General Acceptance Principles

2.1 Canon Compliance Rule

The implementation is acceptable only if it complies with the final canon package.

Compliance means:

- accepted input matches Document 04
- calculations match Document 05
- mappings match Document 06
- output matches Document 07
- versioning and determinism behavior match Document 08
- test behavior matches Document 10

No implementation may be accepted on the basis of:

- being approximately correct
- matching a third party platform
- being operationally convenient
- being close to the intended result
- appearing logically reasonable

2.2 No Interpretation Rule

If implementation required interpretation where the canon was silent, the implementation is not automatically acceptable.

Instead:

- the ambiguity must be recorded
- the affected scope must be paused
- the canon must be clarified
- the implementation must be re aligned to the clarified canon

2.3 Acceptance Scope Rule

Acceptance applies only to the scope explicitly supported in the canon package.

No out of scope behavior may be counted as implementation value for v1 acceptance.

Extra features do not offset canon defects.

---

3. Required Deliverables

3.1 Core Deliverables

The engineer must deliver at minimum:

1. Trinity Engine v1 implementation
2. deterministic execution entry point
3. pinned dependency specification
4. testable build and run instructions
5. golden test execution support
6. implementation notes for any canon sensitive areas
7. versioned release identifier for the delivered build

3.2 Implementation Artifact Requirements

The implementation artifact must include:

- complete source code for the v1 scope
- pinned dependency definitions
- deterministic configuration where relevant
- clear execution instructions
- no missing proprietary black box dependency that prevents review

3.3 Execution Model Requirement

The delivered implementation must match the agreed execution model for v1:

- deterministic core library
- thin CLI test runner or equivalent agreed execution surface

If the execution model differs from the canon aligned implementation agreement, that is a delivery deviation and must be reviewed explicitly.

3.4 Build and Run Instructions

The engineer must provide clear instructions sufficient to:

- install dependencies
- run the implementation
- execute golden tests
- reproduce baseline outputs
- verify version information

These instructions must be specific enough that a reviewer can execute them without relying on undocumented assumptions.

---

4. Required Evidence for Review

4.1 Evidence Package

The engineer must provide an evidence package sufficient to assess canon compliance.

At minimum, this must include:

- implementation version
- dependency versions
- declared canon version target
- declared mapping version target
- declared input schema version target
- golden test results
- any known deviations, blockers, or open issues

4.2 Golden Test Evidence

The evidence package must show:

- which golden cases were executed
- pass or fail status for each case
- any failing cases
- explanation for failures if present

4.3 Defect Disclosure Rule

Known defects must be disclosed explicitly.

The engineer must not present the implementation as fully complete if:

- known canon deviations exist
- known failing fixtures exist
- known ambiguous behaviors remain unresolved
- any required scope item is stubbed, approximated, or bypassed

4.4 No Silent Omission Rule

A missing required feature, rule, mapping, or output field must be disclosed as missing.

It must not be hidden behind:

- partial support language
- implementation shortcuts
- test exclusions without explanation
- informal claims that it will be easy to add later

---

5. Review Criteria

5.1 Input Compliance Review

The implementation must be reviewed against Document 04 to confirm:

- only canonical input is accepted
- rejected input is rejected correctly
- strict numeric typing is enforced
- no silent coercion occurs
- no intake normalization behavior leaks into the engine

5.2 Calculation Compliance Review

The implementation must be reviewed against Document 05 to confirm:

- deterministic calculation order is respected
- node policy is correct by domain
- design time derivation uses the canon method
- gate and line derivation use the pinned rules
- Human Design structural derivations match the canon
- Gene Keys derivation matches the canon

5.3 Mapping Compliance Review

The implementation must be reviewed against Document 06 to confirm:

- the pinned gate order list is used
- the pinned channel table is used
- the pinned center identifiers are used
- no alias drift exists
- the canonical anchor and interval ownership rules are respected

5.4 Output Compliance Review

The implementation must be reviewed against Document 07 to confirm:

- output shape matches exactly
- field names match exactly
- ordering rules match exactly
- numeric precision matches exactly
- datetime formatting matches exactly
- no extra canonical looking fields are invented

5.5 Determinism and Versioning Review

The implementation must be reviewed against Document 08 to confirm:

- required version metadata is present
- no hidden state affects results
- the implementation respects canon precedence
- behavioral changes are not introduced silently

5.6 Golden Test Review

The implementation must be reviewed against Document 10 to confirm:

- required golden cases pass
- no failed cases are ignored
- no fuzzy equivalence is used to justify differences
- expected results match exactly where canon requires exact matching

---

6. Acceptance Conditions

6.1 Minimum Acceptance Conditions

The Trinity Engine v1 implementation may be accepted only if all of the following are true:

1. All required in scope features are implemented.
2. No canon critical ambiguity remains unresolved in the delivered scope.
3. Required version metadata is present and correct.

4. Required golden test cases pass.
5. Required output structure matches the canon exactly.
6. No known canon deviation remains undisclosed.
7. No out of scope shortcuts are being used to simulate compliance.

6.2 Partial Acceptance Rule

Partial acceptance is allowed only if:

- the missing or defective areas are explicitly identified
- the affected scope is narrow and isolated
- the missing scope is not falsely represented as complete
- the remaining accepted scope is genuinely usable and canon compliant

Partial acceptance must never be used to disguise a fundamentally incomplete or ambiguous implementation.

6.3 Provisional Acceptance Rule

If provisional acceptance is ever used, it must state explicitly:

- what is accepted
- what is not accepted
- what blocking items remain
- what evidence is still required

No informal “looks good enough” acceptance is allowed.

---

7. Rejection Conditions

7.1 Automatic Rejection Conditions

The implementation must be rejected for the affected scope if any of the following is true:

- canon critical behavior is missing
- canon critical behavior is approximated without approval
- required mappings are not pinned correctly
- required output fields are missing
- required ordering is wrong
- required precision or formatting is wrong
- required test cases fail
- the engineer resolved ambiguity through interpretation instead of canon escalation
- required deterministic metadata is missing
- hidden non canon behavior affects output

7.2 Rejection Without Debate Rule

The following are sufficient rejection reasons by themselves:

- wrong gate assignment
- wrong line assignment
- wrong design time method
- wrong node policy by domain
- wrong channel activation logic
- wrong Type or Authority derivation
- wrong output contract
- acceptance of invalid engine boundary input

These are not “small differences.” They are core canon failures.

---

8. Defect Handling During Review

8.1 Defect Classification Rule

Review findings should be classified as one of:

- `canon_defect`
- `implementation_defect`
- `fixture_defect`
- `documentation_defect`

8.2 Canon Defect

A `canon_defect` exists when the canon itself is incomplete, conflicting, or insufficiently explicit.

In this case:

- implementation for the affected scope must pause
- the issue must be recorded in Document 09
- the canon must be clarified before final acceptance

8.3 Implementation Defect

An `implementation_defect` exists when the canon is clear but the delivered behavior does not match it.

In this case:

- the implementation must be corrected

- tests must be rerun
- acceptance must be withheld for the affected scope until corrected

8.4 Fixture Defect

A `fixture_defect` exists when a golden test fixture is wrong relative to the canon.

In this case:

- the canon governs
- the fixture must be corrected
- the correction must be documented

8.5 Documentation Defect

A `documentation_defect` exists when implementation may be correct but delivery documentation is incomplete, unclear, or insufficient for reproducible review.

In this case:

- documentation must be corrected before final acceptance

---

9. Communication and Escalation Rules

9.1 Canon Conflict Escalation

If the engineer encounters a canon conflict during implementation:

- the affected scope must be paused
- the conflict must be reported explicitly
- the issue must be recorded in Document 09
- implementation must not proceed by assumption in the affected area

9.2 Clarification Request Rule

Clarification requests are valid only when the canon is genuinely unclear.

They must not be used to reopen already pinned decisions casually.

9.3 No Backchannel Override Rule

No canon behavior is changed by:

- email wording alone
- call notes alone
- chat memory alone

- verbal agreement alone

Any agreed change must be folded back into the governing canon documents before it becomes authoritative.

---

10. Final Acceptance Output

10.1 Acceptance Record

Final acceptance should be documented in a short acceptance record that states:

- accepted implementation version
- accepted canon version
- accepted source_stack_version
- accepted input_schema_version
- accepted mapping_version
- date of acceptance
- any explicit exclusions or reservations if applicable

10.2 Acceptance Statement Rule

The final acceptance statement must not claim more than what was actually verified.

If some scope was not tested or not accepted, that must be stated explicitly.

---

11. Out of Scope for Acceptance

The following do not form part of Trinity Engine v1 acceptance unless explicitly added elsewhere:

- frontend intake experience
- portal rendering
- content generation
- translations
- narrative layer quality
- UX polish
- external product workflows
- non canonical analytics or reporting layers

These may matter to the product, but they are not part of engine acceptance unless separately contracted.

Appendix A

Acceptance Review Checklist

Purpose

This checklist provides an operational review layer for assessing whether the delivered Trinity Engine v1 implementation is acceptable for handoff and approval.

It does not replace the canon package.
It exists to ensure the reviewer checks the actual implementation against the canon in a disciplined and repeatable way.

An item may be marked as:

- `pass`
- `fail`
- `not_applicable`

No item may be treated as passed merely because the implementation appears broadly correct.

A1. Delivery Package Completeness

A1.1 Source Code Delivery

- Full source code for Trinity Engine v1 has been delivered
- No required implementation component is missing
- No black box dependency prevents meaningful review

A1.2 Dependency and Build Delivery

- Dependency definitions are included
- Dependency versions are pinned
- Build instructions are included
- Run instructions are included
- Golden test execution instructions are included

A1.3 Version Declaration

- Delivered implementation version is declared
- Target `canon_version` is declared
- Target `source_stack_version` is declared

- Target `input_schema_version` is declared
- Target `mapping_version` is declared

A2. Input Contract Compliance

A2.1 Canonical Input Shape

- Engine accepts only the canonical input fields:
  - `birth_date`
  - `birth_time`
  - `timezone`
  - `latitude`
  - `longitude`

A2.2 Required Field Enforcement

- Missing required fields are rejected
- Missing field cases are classified correctly as `incomplete_input`

A2.3 Format Enforcement

- `birth_date` requires canonical `YYYY-MM-DD`
- `birth_time` requires canonical `HH:MM`
- `timezone` requires IANA format
- `latitude` requires numeric decimal degrees
- `longitude` requires numeric decimal degrees

A2.4 Strict Type Enforcement

- Latitude provided as string is rejected
- Longitude provided as string is rejected
- No silent numeric coercion occurs

A2.5 Boundary Scope Enforcement

- Engine does not perform birthplace resolution
- Engine does not perform timezone determination
- Engine does not accept raw birthplace text
- No intake normalization logic leaks into the engine

A3. Calculation Canon Compliance

A3.1 Shared Pre Calculation Logic

- Input validation occurs before calculation
- Local datetime is constructed correctly
- UTC conversion is performed correctly
- Julian Day or equivalent canonical astronomical time input is derived correctly
- Input coordinates are used exactly as provided

A3.2 Astrology Calculation

- Tropical zodiac is used
- Placidus house system is used
- Mean Node is used for Astrology output
- Chiron is included
- Earth is derived by canonical opposition logic
- House cusps are calculated and returned
- ASC and MC are calculated and returned
- Unsupported astrology constructs are not emitted

A3.3 Human Design Snapshot Logic

- Personality snapshot is calculated at birth moment
- Design snapshot is calculated at Design time
- Design time uses the canon search method
- Design time does not use forbidden shortcuts
- Personality and Design activations remain separate

A3.4 Gate and Line Logic

- Longitude normalization follows canonical rules
- Gate assignment uses canonical gate width
- Line assignment uses canonical line width
- Half open interval ownership is respected
- Boundary values resolve into the next segment correctly

A3.5 Human Design Structural Logic

- Channels are derived only from canonical channel pairs
- Centers are derived from active channels only
- Definition class is assigned correctly
- Type is assigned by canonical priority rules
- Authority is assigned by canonical priority rules
- Profile is derived correctly from Personality Sun line and Design Sun line
- Incarnation Cross structural encoding is correct

A3.6 Gene Keys Logic

- Gene Keys derives strictly from Human Design outputs
- Activation Sequence positions are mapped correctly
- Key equals corresponding Human Design gate
- Line equals corresponding Human Design line
- No Gene Keys prose or semantic content is emitted

A4. Mapping Canon Compliance

A4.1 Gate Mapping Assets

- Canonical gate sequence is implemented exactly
- Canonical angular anchor is implemented exactly
- Gate order is not procedurally inferred
- No alternate gate sequence is used

A4.2 Channel and Center Mapping

- Canonical channel table is implemented exactly
- Canonical center identifiers are implemented exactly
- Channel to center mapping matches the canon exactly
- Motor center set matches the canon exactly

A4.3 Identifier Hygiene

- No alias drift exists in identifiers
- `ego` is used as the canonical center identifier
- `will` is not emitted as a separate engine identifier
- Gene Keys identifiers match Human Design identifiers exactly

---

A5. Output Contract Compliance

A5.1 Success Response Structure

- Success response contains:
  - `status`
  - `metadata`
  - `input_echo`
  - `astrology`
  - `human_design`
  - `gene_keys`

A5.2 Metadata

- `engine_version` is present
- `canon_version` is present
- `source_stack_version` is present
- `input_schema_version` is present
- `mapping_version` is present
- Metadata contains no non canonical operational junk

A5.3 Input Echo

- `input_echo` matches the canonical normalized input
- No excluded upstream fields appear
- No mutated values appear

A5.4 Astrology Output

- `system` section is correct
- `angles` section is correct
- `house_cusps` section is correct
- `objects` section is correct
- Astrology object order is exact
- Sign identifiers are canonical
- Longitude values use canonical numeric precision

A5.5 Human Design Output

- `system` section is correct
- `personality_activations` order is exact
- `design_activations` order is exact
- `channels` output is correct
- `centers` output is correct
- `definition` value is correct
- `type` value is correct
- `authority` value is correct
- `profile` format is correct
- `incarnation_cross` structural encoding is correct

A5.6 Gene Keys Output

- `system` section is correct
- `activations` section is correct
- `life_work` is correct
- `evolution` is correct

- `radiance` is correct
- `purpose` is correct
- No out of scope Gene Keys content appears

A5.7 Formatting Rules

- Longitude output is numeric and rounded to 6 decimal places
- `design_time_utc` is ISO 8601 UTC with trailing `Z`
- `design_time_utc` uses whole second precision
- Ordered arrays follow canonical order exactly

A6. Error Handling Compliance

A6.1 Error Response Shape

- Error responses contain:
  - `status`
  - `metadata`
  - `error`

A6.2 Error Types

- `invalid_input` is used correctly
- `incomplete_input` is used correctly
- `unsupported_input` is used correctly
- `canon_conflict` is used correctly
- `execution_failure` is used correctly

A6.3 Error Output Discipline

- Error messages are machine readable
- No user facing UI prose appears
- No non canonical stack traces appear in canonical output

A7. Determinism and Versioning Compliance

A7.1 Deterministic Behavior

- Same input plus same version context produces same result
- No hidden state affects outputs
- No non pinned defaults affect outputs

A7.2 Version Governance

- Version metadata is consistent with the delivered implementation
- No silent behavioral changes are present
- Behavioral changes are documented properly if applicable

A7.3 Canon Precedence Compliance

- No implementation choice contradicts canon precedence order
- No ambiguity was resolved by developer interpretation without escalation

---

A8. Golden Test Compliance

A8.1 Baseline Cases

- Minimum valid baseline case count is satisfied
- Baseline full output case passes

A8.2 Edge Cases

- Minimum valid edge case count is satisfied
- Boundary sensitive cases pass
- Wraparound cases pass

A8.3 Invalid and Incomplete Cases

- Minimum invalid input case count is satisfied
- Minimum incomplete input case count is satisfied
- Minimum unsupported input case count is satisfied

A8.4 Regression Sentinel Cases

- Minimum regression sentinel case count is satisfied
- Sentinel cases covering input strictness pass
- Sentinel cases covering mapping boundary behavior pass
- Sentinel cases covering Human Design structural logic pass

A8.5 Exact Match Discipline

- No fuzzy equivalence was used to justify a pass
- Failed fixtures were not ignored
- Any fixture corrections were documented explicitly

A9. Defect and Ambiguity Review

A9.1 Known Defects

- All known implementation defects are disclosed
- All known documentation defects are disclosed
- All known fixture defects are disclosed
- No canon critical issue is hidden

A9.2 Canon Ambiguities

- Any canon ambiguity encountered during implementation is recorded in Document 09
- No canon silent interpretation remains in accepted scope

A10. Acceptance Decision

A10.1 Full Acceptance

Mark full acceptance only if:

- all required in scope features are implemented
- all canon critical checks pass
- required golden tests pass
- no unresolved canon critical ambiguity remains
- no undisclosed deviation remains

A10.2 Partial Acceptance

Mark partial acceptance only if:

- accepted scope is explicitly stated
- rejected or pending scope is explicitly stated
- remaining blockers are explicitly stated
- incomplete scope is not represented as complete

A10.3 Rejection

Reject the implementation if any of the following is true:

- canon critical behavior is wrong
- required mappings are wrong
- required output structure is wrong
- required deterministic metadata is missing
- required golden cases fail
- ambiguity was resolved through unapproved interpretation

- invalid boundary input is silently accepted

A11. Final Acceptance Record

Complete this section at review close:

- **Implementation version reviewed:**
- **Canon version reviewed:**
- **Source stack version reviewed:**
- **Input schema version reviewed:**
- **Mapping version reviewed:**
- **Review outcome:** Full Acceptance / Partial Acceptance / Rejected
- **Date:**
- **Reviewer:**
- **Notes:**